<<enumeration>>
EstadoBoleta
pendiente
pagada
vencida
noCreada
<<enumeration>>
BoletaTipo
inicio
finalizacion
desconocido
BoletaEntity
- id: int
- monto: double
- caratula: String
- gastosAdministrativos: double?
- cuij: String?
- anioExpediente: int?
- nroExpediente: int?
- importePago: double?
- fechaPago: DateTime?
- idBoletaAsociada: int?
- codBarra: String?
- fechaVencimiento: DateTime
- fechaImpresion: DateTime
+ fromJson(json: Map<String, dynamic): BoletaEntity
+ fromCrearBoletaResponse(json: Map<String, dynamic): BoletaEntity
BoletasState
- boletas: List<BoletaEntity>
- currentPage: int
- lastPage: int
- total: int
- perPage: int
- lastSyncTime: DateTime?
- isOfflineData: bool
- hasPreviousPage: bool
- hasNextPage: bool
+ copyWith()
HistorialBoletasScreen
BoletaHistorialModel
- idBoletaGenerada: String
- monto: String
- caratula: String
- idTipoTransaccion: String?
- fechaImpresion: String?
- gastosAdministrativos: String?
- codBarra: String?
- idTipoBoleta: String?
- fechaPago: String?
- diasVencimiento: String?
- estado: String?
+ fromJson(json: Map<String, dynamic>): BoletaHistorialModel
BoletaInicioPagadaModel
- id: String
- fechaImpresion: String
- diasVencimiento: String
- monto: String
- fechaPago: String
- caratula: String
+ fromJson(json: Map<String, dynamic>): BoletaInicioPagadaModel
BoletasNotifier
+ obtenerBoletasCreadas(page: int?, forceRefresh: bool?): void
+ crearBoletaInicio(boletaInicioData: BoletaInicioData): Future<BoletaEntity>
+ crearBoletaFin(boletaFinData: BoletaFinDataState): Future<BoletaEntity>
+ buscarBoletasInicioPagadas(page: int?, caratulaBuscada: String?): Future<void>
+ irAPagina(page: int): Future<void>
+ irAPaginaSiguiente(): Future<void>
+ irAPaginaAnterior(): Future<void>
+ refresh(): Future<void>
- convertirBoletaHistorialAEntity(model: BoletaHistorialModel): BoletaEntity
- convertirBoletaInicioPagadaAEntity(model: BoletaInicioPagadaModel): BoletaEntity
- getFromLocalCache(page: int): Future<BoletasState>
- syncCache(): Future<void>
- handleConnectivityChange(previous: ConnectivityStatus?, current: ConnectivityStatus): void
GenerarBoletaFinalizacionUseCase
+ execute(idBoletaInicio: int, monto: double, fechaRegulacion: DateTime, honorarios: double, caratula: String, cantidadJus: double, valorJus: double, nroExpediente: int?, anioExpediente: int?, cuij: String?): Future<BoletaEntity>
GenerarBoletaInicioUseCase
+ execute(caratula: String, monto: double?): Future<BoletaEntity>
BuscarBoletasInicioPagadasUseCase
+ execute(page: int?, caratulaBuscada: String?): Future<PaginatedResponseModel>
ObtenerHistorialBoletasUseCase
+ execute(page: int?): Future<HistorialBoletasSuccessResponse>
<<Interface>>
BoletasRepository
+ crearBoletaInicio(caratula: String, nroAfiliado: int, monto: double): Future<BoletaEntity>
+ crearBoletaFinalizacion(nroAfiliado, caratula, idBoletaInicio, monto, fechaRegulacion, honorarios, cantidadJus, valorJus, nroExpediente, anioExpediente, cuij): Future<BoletaEntity>
+ obtenerHistorialBoletas(nroAfiliado: int, page: int?): Future<HistorialBoletasSuccessResponse>
+ buscarBoletasInicioPagadas(nroAfiliado: int, page: int?, caratulaBuscada: String?): Future<PaginatedResponseModel>
<<enumeration>>
ConnectivityStatus
online
offline
ConnectivityNotifier
- connectivitySubscription: StreamSubscription<List<ConnectivityResult>>
- checkInternetConnection(): Future<bool>
BoletaFinDataNotifier
+ updateIdBoletaInicio(idBoletaInicio: int): void
+ updateCaratula(updateCaratula: String): void
+ updateExpediente(expediente: int): void
+ updateAnio(anio: int): void
+ updateCuij(cuij: String): void
+ updateFechaRegulacion(fechaRegulacion: DateTime): void
+ updateCantidadJus(cantidadJus: double): void
+ updateValorJus(valorJus: double): void
+ updateHonorarios(honorarios: double): void
+ updateMontoValido(montoValido: double): void
+ reset(): void
BoletasRepositoryImpl
+ crearBoletaInicio(caratula: String, nroAfiliado: int, monto: double): Future<BoletaEntity>
+ crearBoletaFinalizacion(nroAfiliado, caratula, idBoletaInicio, monto, fechaRegulacion, honorarios, cantidadJus, valorJus, nroExpediente, anioExpediente, cuij): Future<BoletaEntity>
+ obtenerHistorialBoletas(nroAfiliado: int, page: int?): Future<HistorialBoletasSuccessResponse>
+ buscarBoletasInicioPagadas(nroAfiliado: int, page: int?, caratulaBuscada: String?): Future<PaginatedResponseModel>
PaginatedResponseModel
- statusCode: int
- data: List<Map<String, dynamic>>
- currentPage: int
- lastPage: int
- total: int
- perPage: int
+ fromJson(json: Map<String, dynamic>): PaginatedResponseModel
BoletaInicioDataNotifier
+ updateActor(actor: String): void
+ updateDemandado(demandado: String): void
+ updateCausa(causa: String): void
+ reset(): void
Paso3BoletaInicioScreen
Paso2BoletaInicioScreen
Paso1BoletaInicioScreen
BoletaStepperWidget
CrearBoletaScreen
MainNavigationScreen
Paso3BoletaFinScreen
Paso2BoletaFinScreen
Paso1BoletaFinScreen
BoletaInicioDataState
- actor: String?
- demandado: String?
- causa: String?
+ isValid(): bool
+ copyWith()
NavigationNotifier
- state: int
- selectTab(index: int): void
BoletaFinDataState
- idBoletaJuicio: int?
- caratula: String?
- expediente: int?
- anio: int?
- cuij: String?
- fechaRegulacion: DateTime?
- cantidadJus: double?
- valorJus: double?
- honorarios: double?
- montoValido: double?
+ isValid(): bool
+ copyWith()
<<abstract>>
CrearBoletaResponse
+ statusCode: int
BoletasDataSource
- client: HttpClient
+ crearBoletaInicio(caratula: String, dato1: String): Future<CrearBoletaResponse>
+ crearBoletaFinalizacion(id_boleta_inicio: int, dato1: String): Future<CrearBoletaResponse>
+ obtenerHistorialBoletas(nroAfiliado: int, page: int?, mostrarPagadas: int): Future<HistorialBoletasResponse>
+ buscarBoletasInicioPagadas(nroAfiliado: int, page: int?, caratulaBuscada: String?): Future<PaginatedResponseModel>
BoletasLocalDataSource
- client: HttpClient
+ guardarBoletas(boletas: List<BoletaEntity>): void
+ obtenerBoletasLocales(limit: int?, offset: int?, caratulaFiltro: String?): Future<List<BoletaEntity>>
+ obtenerConteoBoletasLocales(caratulaFiltro: String?): Future<int>
+ limpiarCache(): void
+ obtenerUltimaSincronizacion(): Future<Datetime?>
+ tieneBoletasEnCache(): Future<bool>
+ boletaToMap(boleta: BoletaEntity): Map<String, dynamic>
+ mapToBoleta(map: Map<String, dynamic>): BoletaEntity
+ stringToBoletaTipo(tipo: String): BoletaTipo

<<abstract>>
CrearBoletaResponse
+ statusCode: int
Extends
Extends
CrearBoletaGenericErrorResponse
+ statusCode: int
+ errorMessage: String
+ fromJson(statusCode: int, json: Map<String, dynamic>): CrearBoletaGenericErrorResponse
+ toJson(): Map<String, dynamic>
CrearBoletaSuccessResponse
+ statusCode: int
+ success: bool
+ fromJson(statusCode: int, json: Map<String, dynamic>): CrearBoletaSuccessResponse
+ toJson(): Map<String, dynamic>